

[Skip to main content](#)

ClaudeAI ×

Search in r/ClaudeAI



Create

r/ClaudeAI • 6mo ago
yksugi

25 Claude Code Tips from 11 Months of Intense Use

Productivity

[My previous post with 10 tips](#) was well-received, so I decided to expand it to 25 here.

The GitHub repo: <https://github.com/ykdojo/claude-code-tips>

Tip 0: Customize your status line

You can customize the status line at the bottom of Claude Code to show useful info. I set mine up to show the model, current directory, git branch (if any), uncommitted file count, sync status with origin, and a visual progress bar for token usage. It also shows a second line with my last message so I can see what the conversation was about:

```
Opus 4.5 | 📁 claude-code-tips | 📁 main (scripts/context-bar.sh uncommitted, synced 12m  
💬 This is good. I don't think we need to change the documentation as long as we don't
```

This is especially helpful for keeping an eye on your context usage and remembering what you were working on. The script also supports 10 color themes (orange, blue, teal, green, lavender, rose, gold, slate, cyan, or gray).

To set this up, you can use [this sample script](#) and check the [setup instructions](#).

Tip 1: Learn a few essential slash commands

There are a bunch of built-in slash commands (type `/` to see them all). Here are a few worth knowing:

/usage

Check your rate limits:

Current session



14% used

Resets 3:59pm (Asia/Tokyo)

Current week (all models)



26% used

Resets Jan 3, 2026, 5:59am (Asia/Tokyo)

If you want to watch your usage closely, keep it open in a tab and use Tab then Shift+Tab or ← then → to refresh.

[Join the conversation](#)

/chrome

Skip to main content



Create



```
> /chrome
Chrome integration enabled
```

/mcp

Manage MCP (Model Context Protocol) servers:

Manage MCP servers

1 server

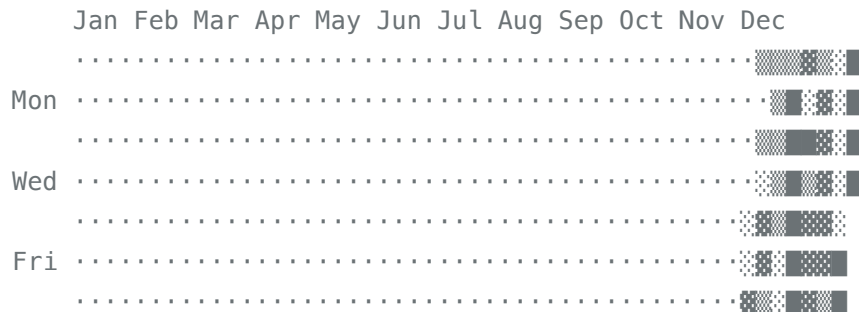
› 1. playwright ✓ connected · Enter to view details

MCP Config locations (by scope):

- User config (available in all your projects):
 - /Users/yk/.claude.json

/stats

View your usage statistics with a GitHub-style activity graph:



Less More

Favorite model: Opus 4.5

Total tokens: 12.1m

Sessions: 1.8k

Longest session: 20h 40m 45s

Current streak: 44 days

Longest streak: 45 days

Active days: 49/51

Peak hour: 17:00–18:00

You've used ~145x more tokens than Brave New World

/clear

Clear the conversation and start fresh.

Tip 2: Talk to Claude Code wit

Skip to main content



Create



On my Mac, I've tried a few different options:

- superwhisper
- MacWhisper
- [Super Voice Assistant](#)

You can get more accuracy by using a hosted service, but I found that a local model is strong enough for this purpose. Even when there are mistakes or typos in the transcription, Claude is smart enough to understand what you're trying to say. Sometimes you need to say certain things extra clearly, but overall local models work well enough.

For example, Claude was able to interpret mistranscribed words like "ExcelElanishMark" and "advast" correctly as "exclamation mark" and "Advanced".

A common objection is "what if you're in a room with other people?" I just whisper using earphones - I personally like Apple EarPods (not AirPods). They're affordable, high quality enough, and you just whisper into them quietly. I've done it in front of other people and it works well. In offices, people talk anyway - instead of talking to coworkers, you're talking quietly to your voice transcription system. I don't think there's any problem with that. This method works so well that it even works on a plane. It's loud enough that other people won't hear you, but if you speak close enough to the mic, your local model can still understand what you're saying. (In fact, I'm writing this very paragraph using that method on a flight.)

Tip 3: Break down large problems into smaller ones

This is one of the most important concepts to master. It's exactly the same as traditional software engineering - the best software engineers already know how to do this, and it applies to Claude Code too.

If you find that Claude Code isn't able to one-shot a difficult problem or coding task, ask it to break it down into multiple smaller issues. See if it can solve an individual part of that problem. If it's still too hard, see if it can solve an even smaller sub-problem. Keep going until everything is solvable.

Essentially, instead of going from A to B directly, you can go from A to A1 to A2 to A3, then to B.

A good example of this is when I was building my own voice transcription system. I needed to build a system that could let the user select and download a model, take keyboard shortcuts, start transcribing, put the transcribed text at the user's cursor, and wrap all of this in a nice UI. That's a lot. So I broke it down into smaller tasks. First, I created an executable that would just download a model, nothing else. Then I created another one that would just record voice, nothing else. Then another one that would just transcribe pre-recorded audio. I completed them one by one like that before combining them at the end.

Highly related to this: your problem-solving skills and software engineering skills are still highly relevant in the world of agentic coding and Claude Code. It's able to solve a lot of problems on its own, but when you apply your general problem-solving and software engineering skills to it, it becomes a lot more powerful.

Tip 4: Using Git and GitHub CLI like a pro

Just ask Claude to handle your Git and GitHub CLI tasks. This includes committing (so you don't have to write commit messages manually), branching, pulling

Skip to main content



Create



For GitHub CLI (`gh`), there's a lot you can do. One thing I started doing more after using Claude Code is creating draft PRs. This lets Claude Code handle the PR creation process with low risk - you can review everything before marking it ready for review.

And it turns out, `gh` is pretty powerful. You can even send arbitrary GraphQL queries through it. For example, you can even find the exact times at which GitHub PR descriptions were edited:

- Bash(`gh api graphql -f query='`

```

query {
  repository(owner: "...", name: "...") {
    pullRequest(number: ...) {
      userContentEdits(first: 100) {
        nodes { editedAt editor { login } }
      }
    }
  }
}
```

`'')`
- Here's the full edit history for your PR description:

#	Edited At (UTC)	Editor
1	2025-12-01 00:08:34	ykdojo
2	2025-12-01 15:57:21	ykdojo
3	2025-12-01 16:24:33	ykdojo
4	2025-12-01 16:27:00	ykdojo
5	2025-12-04 00:40:02	ykdojo
...		

Tip 5: AI context is like milk; it's best served fresh and condensed!

When you start a new conversation with Claude Code, it performs the best because it doesn't have all the added complexity of having to process the previous context from earlier parts of the conversation. But as you talk to it longer and longer, the context gets longer and the performance tends to go down.

So it's best to start a new conversation for every new topic, or if the performance starts to go down.

Tip 6: Getting output out of your terminal

Sometimes you want to copy and paste Claude Code's output, but copying directly from the terminal isn't always clean. Here are a few ways to get content out more easily:

- **Clipboard directly:** On Mac or Linux, ask Claude to use `pbcopy` to send output straight to your clipboard
- **Write to a file:** Have Claude put the content in a (file or editor) so you can copy from there. You can also

Skip to main content



Create



- **Opening URLs:** If there's a URL you want to examine yourself, ask Claude to open it in your browser. On Mac, you can ask it to use the `open` command, but in general asking to open in your favorite browser should work on any platform
- **GitHub Desktop:** You can ask Claude to open the current repo in GitHub Desktop. This is particularly useful when it's working in a non-root directory - for example, if you asked it to create a git worktree in a different directory and you haven't opened Claude Code from there yet

You can combine some of these together too. For example, if you want to edit a GitHub PR description, instead of having Claude edit it directly (which it might mess up), you can have it copy the content into a local file first. Let it edit that, check the result yourself, and once it looks good, have it copy and paste it back into the GitHub PR. That works really well. Or if you want to do that yourself, you can just ask it to open it in VS Code or give it to you via pbcopy so you can copy and paste it manually.

Of course, you can run these commands yourself, but if you find yourself doing it repetitively, it's helpful to let Claude run them for you.

Tip 7: Set up terminal aliases for quick access

Since I use the terminal more because of Claude Code, I found it helpful to set up short aliases so I can launch things quickly. Here are the ones I use:

- `c` for Claude Code (this is the one I use the most)
- `ch` for Claude Code with Chrome integration
- `gb` for GitHub Desktop
- `co` for VS Code
- `q` for going to the project directory where I have most projects. From there I can manually `cd` into an individual folder to work on that project, or I can just launch Claude Code with `c` to let it basically have access to any project it needs to access.

To set these up, add lines like this to your shell config file (`~/.zshrc` or `~/.bashrc`):

```
alias c='claude'
alias ch='claude --chrome'
alias gb='github'
alias co='code'
alias q='cd ~/Desktop/projects'
```

Once you have these aliases, you can combine them with flags: `c -c` continues your last conversation, and `c -r` shows a list of recent conversations to resume. These work with `ch` too (`ch -c`, `ch -r`) for Chrome sessions.

Tip 8: Proactively compact your context

There's a `/compact` command in Claude Code that : space. Automatic compaction also happens when the context window for Opus 4.5 is currently 200k, and 4

Skip to main content



compaction happens.

The way I do this is to ask Claude to write a handoff document before starting fresh. Something like:

Put the rest of the plan in the system-prompt-extraction folder as HANDOFF.md. Explain what you have tried, what worked, what didn't work, so that the next agent with fresh context is able to just load that file and nothing else to get started on this task and finish it up.

Claude will create a file summarizing the current state of work:

```
• Write(experiments/system-prompt-extraction/HANDOFF.md)
  | Wrote 129 lines to experiments/system-prompt-extraction/HANDOFF.md
  | # System Prompt Slimming – Handoff Document
  | ## Goal
  | Reduce Claude Code's system prompt by ~45% (currently at 11%, need ~34% more).
  | ## Current Progress
  | ### What's Been Done
  | - **Backup/restore system**: `backup-cli.sh` and `restore-cli.sh` with SHA256 veri
  | - **Patch system**: `patch-cli.js` that restores from backup then applies patches
  | ...
```

After Claude writes it, review it quickly. If something's missing, ask for edits:

Did you add a note about iteratively testing instead of trying to do everything all at once?

Then start a fresh conversation. For the fresh agent, you can just give the path of the file and nothing else like this, and it should work just fine:

```
> experiments/system-prompt-extraction/HANDOFF.md
```

In subsequent conversations, you can ask the agent to update the document for the next agent.

I've also created a `/handoff` slash command that automates this - it checks for an existing HANDOFF.md, reads it if present, then creates or updates it with the goal, progress, what worked, what didn't, and next steps. You can find it in the [commands folder](#).

Tip 9: Complete the write-test cycle for autonomous tasks

If you want Claude Code to run something autonomously, like `git bisect`, you need to give it a way to verify results. The key is completing the write-test cycle: write code, run it, check the output, and repeat.

For example, let's say you're working on Claude Code itself and you notice `/compact` stopped working and started throwing a 400 error. A classic tool to find the exact commit that caused this is `git bisect`. The nice thing is you can let Claude Code run bisect

For tasks that involve interactive terminals like Claude

Skip to main content



3. Capture the output

4. Verify it's what you expect

Here's a simple example of testing if `/context` works:

```
tmux kill-session -t test-session 2>/dev/null
tmux new-session -d -s test-session
tmux send-keys -t test-session 'claude' Enter
sleep 2
tmux send-keys -t test-session '/context' Enter
sleep 1
tmux capture-pane -t test-session -p
```

Once you have a test like this, Claude Code can run `git bisect` and automatically test each commit until it finds the one that broke things.

This is also an example of why your software engineering skills still matter. If you're a software engineer, you probably know about tools like `git bisect`. That knowledge is still really valuable when working with AI - you just apply it in new ways.

Another example is simply writing tests. After you let Claude Code write some code, if you want to test it, you can just let it write tests for itself too. And let it run on its own and fix things if it can. Of course, it doesn't always go in the right direction and you need to supervise it sometimes, but it's able to do a surprising amount of coding tasks on its own.

Creative testing strategies

Sometimes you need to be creative with how you complete the write-test cycle. For example, if you're building a web app, you could use Playwright MCP, Chrome DevTools MCP, or Claude's native browser integration (through `/chrome`). I haven't tried Chrome DevTools yet, but I've tried Playwright and Claude's native integration. Overall, Playwright generally works better. It does use a lot of context, but the 200k context window is normally enough for a single task or a few smaller tasks.

The main difference between these two seems to be that Playwright focuses on the accessibility tree (structured data about page elements) rather than taking screenshots. It does have the ability to take screenshots, but it doesn't normally use them to take actions. On the other hand, Claude's native browser integration focuses more on taking screenshots and clicking on elements by specific coordinates. It can click on random things sometimes, and the whole process can be slow.

This might improve over time, but by default I would go with Playwright for most tasks that aren't visually intensive. I'd only use Claude's native browser integration if I need to use a logged-in state without having to provide credentials (since it runs in your own browser profile), or if it specifically needs to click on things visually using their coordinates.

This is why I disable Claude's native browser integration by default and use it through the `ch` shortcut I defined previously. That way Playwright handles most browser tasks, and I only enable Claude's native integration when I specifically need it.

[Skip to main content](#) **Create** ²

```
# Claude for Chrome
```

- Use ``read_page`` to get element refs from the accessibility tree
- Use ``find`` to locate elements by description
- Click/interact using ``ref``, not coordinates
- NEVER take screenshots unless explicitly requested by the user

In my personal experience, I've also had a situation where I was working on a Python library ([Daft](#)) and needed to test a version I built locally on Google Colab. The trouble is it's hard to build a Python library with a Rust backend on Google Colab - it doesn't seem to work that well. So I needed to actually build a wheel locally and then upload it manually so that I could run it on Google Colab. I also tried monkey patching, which worked well in the short term before I had to wait for the whole wheel to build locally. I came up with these testing strategies and executed them by going back and forth with Claude Code.

Another situation I encountered is I needed to test something on Windows but I'm not running a Windows machine. My CI tests on the same repo were failing because we had some issues with Rust on Windows, and I had no way of testing locally. So I needed to create a draft PR with all the changes, and another draft PR with the same changes plus enabling Windows CI runs on non-main branches. I instructed Claude Code to do all of that, and then I tested the CI directly in that new branch.

Tip 10: Cmd+A and Ctrl+A are your friends

I've been saying this for a few years now: Cmd+A and Ctrl+A are friends in the world of AI. This applies to Claude Code too.

Sometimes you want to give Claude Code a URL, but it can't access it directly. Maybe it's a private page (not sensitive data, just not publicly accessible), or something like a Reddit post that Claude Code has trouble fetching. In those cases, you can just select all the content you see (Cmd+A on Mac, Ctrl+A on other platforms), copy it, and paste it directly into Claude Code. It's a pretty powerful method.

This works great for terminal output too. When I have output from Claude Code itself or any other CLI application, I can use the same trick: select all, copy, and paste it back to CC. Pretty helpful.

Some pages don't lend themselves well to select all by default - but there are tricks to get them into a better state first. For example, with Gmail threads, click Print All to get the print preview (but cancel the actual print). That page shows all emails in the thread expanded, so you can Cmd+A the entire conversation cleanly.

This applies to any AI, not just Claude Code.

Tip 11: Use Gemini CLI as a fallback for blocked sites

Claude Code's WebFetch tool can't access certain sites, like Reddit. But you can work around this by creating a skill that tells Claude to use Gemini CLI as a fallback. Gemini has web access and can fetch content from sites that Claude can't reach directly.

This uses the same tmux pattern from Tip 9 - start a file goes in `~/ .claude/skills/reddit-fetch/SKILL`

[Skip to main content](#)

Create



simpler, you can put a condensed version in `(~/ .claude/CLAUDE.md)` instead, but that gets loaded into every conversation whether you need it or not.

I tested this by asking Claude Code to check how Claude Code skills are regarded on Reddit - a bit meta. It goes back and forth with Gemini for a while, so it's not fast, but the report quality was surprisingly good. Obviously, you'll need to have Gemini CLI installed for this to work.

Tip 12: Invest in your own workflow

Personally, I've created my own voice transcription app from scratch with Swift. I created my own custom status line from scratch using Claude Code, this one with bash. And I created my own system for simplifying the system prompt in Claude Code's minified JavaScript file.

But you don't have to go overboard like that. Just taking care of your own CLAUDE.md, making sure it's as concise as possible while being able to help you achieve your goals - stuff like that is helpful. And of course, learning these tips, learning these tools, and some of the most important features.

All of these are investments in the tools you use to build whatever you want to build. I think it's important to spend at least a little bit of time on that.

Tip 13: Search through your conversation history

You can ask Claude Code about your past conversations, and it'll help you find and search through them. Your conversation history is stored locally in `(~/ .claude/projects/)`, with folder names based on the project path (slashes become dashes).

For example, conversations for a project at `(/Users/yk/Desktop/projects/claude-code-tips)` would be stored in:

```
~/ .claude/projects/-Users-yk-Desktop-projects-claude-code-tips/
```

Each conversation is a `(.jsonl)` file. You can search through them with basic bash commands:

```
# Find all conversations mentioning "Reddit"
grep -l -i "reddit" ~/ .claude/projects/-Users-yk-Desktop-projects-*/*.jsonl

# Find today's conversations about a topic
find ~/ .claude/projects/-Users-yk-Desktop-projects-*/*.jsonl -mtime 0 -exec grep -l -i '

# Extract just the user messages from a conversation (requires jq)
cat ~/ .claude/projects/.../conversation-id.jsonl | jq -r 'select(.type=="user") | .mess
```

Or just ask Claude Code directly: "What did we talk about regarding X today?" and it'll search through the history for you.

Tip 14: Multitasking with termi

[Skip to main content](#) **Create** ²

My personal method is what I would call a "cascade" - whenever I start a new task, I just open a new tab on the right. Then I sweep left to right, left to right, going from oldest tasks to newest. The general direction stays consistent, except when I need to check on certain tasks, get notifications, etc.

Tip 15: Slim down the system prompt

Claude Code's system prompt and tool definitions take up about 19k tokens (~10% of your 200k context) before you even start working. I created a patch system that reduces this to about 9k tokens - saving around 10,000 tokens (~50% of the overhead).

Component	Before	After	Savings
System prompt	3.0k	1.8k	1,200 tokens
System tools	15.6k	7.4k	8,200 tokens
Total	~19k	~9k	~10k tokens (~50%)

The patches work by trimming verbose examples and redundant text from the minified CLI bundle while keeping all the essential instructions.

I've tested this extensively and it works well. It feels more raw - more powerful, but maybe a little less regulated, which makes sense because the system instruction is shorter. It feels more like a pro tool when you use it this way. I really enjoy starting with lower context because you have more room before it fills up, which gives you the option to continue conversations a bit longer. That's definitely the best part of this strategy.

Check out the [system-prompt folder](#) for the patch scripts and full details on what gets trimmed.

Why patching? Claude Code has flags that let you provide a simplified system prompt from a file (`--system-prompt` or `--system-prompt-file`), so that's another way to go about it. But for the tool descriptions, there's no official option to customize them. Patching the CLI bundle is the only way. Since my patch system handles everything in one unified approach, I'm keeping it this way for now. I might re-implement the system prompt portion using the flag in the future.

Requirements: These patches require npm installation (`npm install -g @anthropic-ai/claude-code`). The patching works by modifying the JavaScript bundle (`cli.js`) - other installation methods may produce compiled binaries that can't be patched this way.

Important: If you want to keep your patched system prompt, disable auto-updates by adding this to `~/.claude/settings.json`:

```
{
  "env": {
    "DISABLE_AUTOUPDATER": "1"
  }
}
```

[Skip to main content](#)[+ Create](#)[🔔²](#)

Lazy-load MCP tools

If you use MCP servers, their tool definitions are loaded into every conversation by default – even if you don't use them. This can add significant overhead, especially with multiple servers configured.

Enable lazy-loading so MCP tools are only loaded when needed:

```
{
  "env": {
    "ENABLE_TOOL_SEARCH": "true"
  }
}
```

Add this to `~/.claude/settings.json`. Claude will search for and load MCP tools on-demand rather than having them all present from the start. As of version 2.1.7, this happens automatically when MCP tool descriptions exceed 10% of the context window.

Tip 16: Git worktrees for parallel branch work

If you're working on multiple files or multiple branches and you don't want them to get conflicted, Git worktrees are a great way to work on them at the same time. You can just ask Claude Code to create a git worktree and start working on it there – you don't have to worry about the specific syntax.

The basic idea is that you can work on a different branch in a different directory. It's essentially a branch + a directory.

You can add this layer of Git worktrees on top of the cascade method I discussed in the multitasking tip.

Tip 17: Manual exponential backoff for long-running jobs

When waiting on long-running jobs like Docker builds or GitHub CI, you can ask Claude Code to do manual exponential backoff. Exponential backoff is a common technique in software engineering, but you can apply it here too. Ask Claude Code to check the status with increasing sleep intervals – one minute, then two minutes, then four minutes, and so on. It's not programmatically doing it in the traditional sense – the AI is doing it manually – but it works pretty well.

This way the agent can continuously check the status and let you know once it's done.

(For GitHub CI specifically, `gh run watch` exists but outputs many lines continuously, which wastes tokens. Manual exponential backoff with `gh run view <run-id> | grep <job-name>` is actually more token-efficient. This is also a general technique that works well even when you don't have a dedicated wait command handy.)

Tip 18: Claude Code as a writi

[Skip to main content](#)

Then I go through it line by line, pretty much. I say okay, let's take a look at it together. I like this line for these reasons. I feel like this line needs to move over there. This line needs to change in this particular way. I might ask about reference materials as well.

So it's this sort of back-and-forth process, maybe with the terminal on the left and your code editor on the right. That tends to work really well.

Tip 19: Markdown is the s**t

Typically when people write a new document, they might use something like Google Docs or maybe Notion. But now I honestly think the most efficient way to go about it is markdown.

Markdown was already pretty good even before AI, but with Claude Code in particular, because it's so efficient as I mentioned with regards to writing, it makes the value of markdown higher in my opinion. Whenever you want to write a blog post or even a LinkedIn post, you can just talk to Claude Code, have it be saved as markdown, and then go from there.

A quick tip for this one: if you want to copy and paste markdown content into a platform that doesn't accept it easily, you can paste it into a fresh Notion file first, then copy from Notion into the other platform. Notion converts it to a format that other platforms can accept. If regular pasting doesn't work, try Command + Shift + V to paste without formatting.

Tip 20: Use Notion to preserve links when pasting

It turns out the reverse also works. If you have text with links from other places, let's say from Slack, you can copy it. If you paste it directly into Claude Code, it doesn't show the links. But if you put it in a Notion document first, then copy from there, you get it in markdown, which of course Claude Code can read.

Tip 21: Containers for long-running risky tasks

Regular sessions are more for methodical work where you control the permissions you give and review output more carefully. Containerized environments are great for `--dangerously-skip-permissions` sessions where you don't have to give permission for each little thing. You can just let it run on its own for a while.

This is useful for research or experimentation, things that take a long time and maybe could be risky. A good example is the Reddit research workflow from Tip 11, where the reddit-fetch skill goes back and forth with Gemini CLI through tmux. Running that unsupervised is risky on your main system, but in a container, if something goes wrong, it's contained.

Another example is how I created the [system prompt patching scripts](#) in this repo. When a new version of Claude Code comes out, I need to update the patches for the minified CLI bundle. Instead of running Claude Code with `--dangerously-skip-permissions` on my host machine (where it has access to everything), I run it in a container. Claude Code can explore the minified JavaScript, find the variable mappings, and create new patch files without me approving every little thing that way.

In fact, it was able to complete the migration pretty n that some didn't work with the new version, iterated i

[Skip to main content](#) Create ²

Check out the [container folder](#) for the Dockerfile and setup instructions.

Advanced: Orchestrating a worker Claude Code in a container

You can take this further by having your local Claude Code control another Claude Code instance running inside a container. The trick is using tmux as the control layer:

1. Your local Claude Code starts a tmux session
2. In that tmux session, it runs or connects to the container
3. Inside the container, Claude Code runs with `--dangerously-skip-permissions`
4. Your outer Claude Code uses `tmux send-keys` to send prompts and `capture-pane` to read output

This gives you a fully autonomous "worker" Claude Code that can run experimental or long-running tasks without you approving every action. When it's done, your local Claude Code can pull the results back. If something goes wrong, it's all sandboxed in the container.

Advanced: Multi-model orchestration

Beyond just Claude Code, you can run different AI CLIs in containers - Codex, Gemini CLI, or others. I tried OpenAI Codex for code review, and it works well. The point isn't that you can't run these CLIs directly on your host machine - you obviously can. The value is that Claude Code's UI/UX is smooth enough that you can just talk to it and let it handle the orchestration: spinning up different models, sending data between containers and your host. Instead of manually switching between terminals and copy-pasting, Claude Code becomes the central interface that coordinates everything.

Tip 22: The best way to get better at using Claude Code is by using it

Recently I saw a world-class rock climber being interviewed by another rock climber. She was asked, "How do you get better at rock climbing?" She simply said, "By rock climbing."

That's how I feel about this too. Of course, there are supplementary things you can do, like watching videos, reading books, learning about tips. But using Claude Code is the best way to learn how to use it. Using AI in general is the best way to learn how to use AI.

I like to think of it like a billion token rule instead of the 10,000 hour rule. If you want to get better at AI and truly get a good intuition about how it works, the best way is to consume a lot of tokens. And nowadays it's possible. I found that especially with Opus 4.5, it's powerful enough but affordable enough that you can run multiple sessions at the same time. You don't have to worry as much about token usage, which frees you up a lot.

Tip 23: Clone and half-clone conversations

Sometimes you want to try a different approach from [original thread](#). The [clone-conversation script](#) lets you can branch off.

Skip to main content



Create



To set it up manually, symlink both files:

```
ln -s /path/to/this/repo/scripts/clone-conversation.sh ~/.claude/scripts/clone-conversation.sh
ln -s /path/to/this/repo/commands/clone.md ~/.claude/commands/clone.md
```

Then just type `/clone` in any conversation and Claude will handle finding the session ID and running the script.

I've tested this extensively and the cloning works really well.

Half-clone to reduce context

When a conversation gets too long, the [half-clone-conversation script](#) keeps only the later half. This reduces token usage while preserving your recent work. The first message is tagged with `[HALF-CLONE <timestamp>]` (e.g., `[HALF-CLONE Jan 7 14:30]`).

To set it up manually, symlink both files:

```
ln -s /path/to/this/repo/scripts/half-clone-conversation.sh ~/.claude/scripts/half-clone-conversation.sh
ln -s /path/to/this/repo/commands/half-clone.md ~/.claude/commands/half-clone.md
```

Recommended permission for clone scripts

Both clone scripts need to read `~/.claude` (for conversation files and history). To avoid permission prompts from any project, add this to your global settings (`~/.claude/settings.json`):

```
{
  "permissions": {
    "allow": ["Read(~/.claude)"]
  }
}
```

Tip 24: Use realpath to get absolute paths

When you need to tell Claude Code about files in a different folder, use `realpath` to get the full absolute path:

```
realpath some/relative/path
```

Tip 25: Understanding [CLAUDE](#) Commands vs Plugins

[Skip to main content](#) [Create](#) ²

CLAUDE.md is the simplest one. It's a bunch of files that get treated as the default prompt, loaded into the beginning of every conversation no matter what. The nice thing about it is the simplicity. You can explain what the project is about in a particular project (`./CLAUDE.md`) or globally (`~/.claude/CLAUDE.md`).

Skills are like better-structured CLAUDE.md files. They can be invoked by Claude automatically when relevant, or manually by the user with a slash (e.g., `/my-skill`). For example, you could have a skill that opens a Google Translate link with proper formatting when you ask how to pronounce a word in a certain language. If those instructions are in a skill, they only load when needed. If they were in CLAUDE.md, they'd already be there taking up space. So skills are more token-efficient in theory.

Slash Commands are similar to skills in that they're ways of packaging instructions separately. They can be invoked manually by the user, or by Claude itself. If you need something more precise, to invoke at the right time at your own pace, slash commands are the tool to use.

Skills and slash commands are pretty similar in the way they function. The difference is the intention of the design - skills are primarily designed for Claude to use, and slash commands are primarily designed for the user to use. However, they have ended up [merging them](#), as I had [suggested this change](#).

Plugins are a way to package skills, slash commands, agents, hooks, and MCP servers together. But a plugin doesn't have to use all of them. Anthropic's official `frontend-design` plugin is essentially just a skill and nothing else. It could be distributed as a standalone skill, but the plugin format makes it easier to install.

(Couldn't post all 40+ tips here because of the character limit. You can see the rest on this GitHub repo: <https://github.com/ykdojo/claude-code-tips>)

569

53

3

Sort by: **Best** ▾ Search Comments**ClaudeAI-mod-bot**  App **MOD** • 6mo ago •  Stickied comment

Wilson, lead ClaudeAI modbot

If this post is showcasing a project you built with Claude, please change the post flair to Built with Claude so that it can be easily found by others.

**champdebloom**  • 6mo ago

If you notice Claude struggles to fetch the content from a page, you can use the [Obsidian Web Clipper](#) to copy the page content as Markdown.

[Skip to main content](#) Create 2**herms14** • 6mo ago

this guide should be pinned on this sub.

**telesteriaq** • 6mo ago • Edited 6mo ago

For me it's about reducing token bloat.

that's actually why a lot of workflows are structured the way they are ATM

Opus 4.5 in Claude Code still struggles with knowing what to keep vs. what to drop once the context window fills up. So you end up designing around that limitation.

As a little side note you can use Claude -c instead of Claude --continue

**glauberlima** • 6mo ago

Thanks for the comprehensive guide!

Speaking of statuslines, I actually built a custom one recently that is very easy to install. I'd love to hear your feedback, reddit.

DOWNLOAD: <https://github.com/glauberlima/claude-code-statusline>

**addiktion** • 6mo ago • Edited 6mo ago Top 1% Commenter

I've been using [Voicelink](#) as well which is a cheaper one time payment over Super Whisper if you are a mac user. I've been pretty impressed with it. Both use the Nvidia Parakeet model locally which is pretty dang fast. It's definitely helped me speed up my prompt inputs.

Now I just need something to speed up my outputs so learning how to read at a faster pace, haha.

**McNemarra** • 6mo ago

Git worktrees is the one thing I'm still figuring out if its useful

**maigpy** • 6mo ago

just use multiple repo clones, one for each fe

[Skip to main content](#) Create ²**karanb192** • 6mo ago

Solid list. One more tip: hooks.

They let you automate things like:

- Block dangerous commands before execution
- Auto-format files after edits
- Get notifications when Claude needs input

Complements your context management tips: karanbansal.in/blog/claude-code-hooks

**AlvsLib** • 5mo ago

Gave the link to this post to Claude Code to set everything up (reddit refused connection but it found the github). It guided me through everything. Truly amazing times.

**spiffco7** • 6mo ago**boopatron** • 6mo ago

I can't use superwhisper etc at work, but parakeet v3 using MLX works amazingly well (and is very fast) locally on a mac. I got claude to write a pretty sweet replacement using it in an hour or two by writing a decent spec and using the ralph loop plugin

**Spiritual_Bunch334** • 6mo ago

Solid list! Tip #4 (breaking down complex problems) is underrated.

One thing I'd add: when working on larger projects, keeping separate context for different concerns (backend logic vs frontend vs testing) helps a lot.

Instead of one long conversation that tries to do everything, shorter focused sessions tend to produce cleaner results.

The fresh conversation tip (#6) is huge - I've noticed Claude performs noticeably better in the first 10-15 exchanges vs later in a lc

Skip to main content **Create** ²

tool, would load it up with more prompt content than necessary?

**telesteriaq** • 6mo ago

Did you actually take a look at it? There is definitely stuff you can trim. It's also the reason Claude uses wrong anthropic model names for openrouter api calls

**gopietz** • 6mo ago

Then tell me why Anthropic doesn't trim it in the first place. Do you think they're interested in shipping junk?

Or consider this: You know less about how Claude needs to be prompted than dozens of the best AI engineers on this planet, who also trained the model.

**telesteriaq** • 6mo ago

On top of my head some stuff I remember:

1. They don't know your exact use case so they preprompt things you won't need.
2. Ethical reject request fluff
3. Anthropic product information
4. Tone and language

Also they literally made blog posts about this lmao and how they have to balance it. So idk who your fighting for xD

**maigpy** • 6mo ago

useful and coherent, thank you.

**evia89** • 6mo ago

Stull like this def helps claude code :D

IMPORTANT: Assist with authorized security testing, defensive security, CTF challenges, and educational contexts. Refuse requests for destructive techniques, DoS attacks, mass targeting, supply chain compromise, or detection evasion for malicious purposes. Dual-use security tools (C2 frameworks, credential testing, exploit development) require clear authorization context: pentesting engagements, CTF competitions, security research, or defensive use cases.

Its like telling human dont think about

Skip to main content

 Create

 ²



loads of context space, and is unnecessary as claude can use the gh cli



gopietz • 6mo ago

No, you're wrong. The numbers from above are from the Claude Code defaults and its internal tools.



Touix • 6mo ago

I recomment Handy for speech to text Its free and open source andd can be fully local It work great !



rageagainistjg • 6mo ago

This is very very helpful. I am just wondering if you have ever used codex and your views on it as a Claude code pro. Would love to hear opinion



vicethal • 6mo ago

I've found tmux capture-pane to be primitive and error-prone, scrolling output and animated elements make it hazardous to rely on. But send-keys works pretty well, and hooks plus having Claude use an MCP server for a message system are pretty robust ways of getting output.



Maleficent_Sun_5597 • 6mo ago

Cool



Substantial-Rub-1240 • 6mo ago

Tip 26! Windows uses can press win + H to speak into terminal ! Just found this out.



chrischen-003 • 6mo ago

yksugi25Claude CodeTip 11GitHub repoClaude CodeTip 5

Gemini CLITip 11Claude Code



Avidium18 • 6mo ago
Expert AI

[Skip to main content](#)

Create

**Acceptable-Kick-7102** • 6mo ago

The main problem with Claude for me are rate limits. Even tokens are not such problem as rate limits. So there is not point of learning all these fancy tricks about tmux sessions, multitasking etc. if you hit those limits eventually.

The only tip which addresses this problem in viable way is Multi-model orchestration. Yes I tried (and im still trying) to use gemini and ollama to delegate some tasks to them (like analyzing building log, or executing some azure deployment tasks) with agents. But Claude still tends to go back to itself and you hit rate limits in no time. Especially when context is long.

Unfortunately there is no subscription like Pro but with higher rate limits (like for 50-60\$ per month). You have to switch to Max which costs 100\$ from the start. Its no go for me. So i'll probably eventually look for another tool.

**Odd-Librarian4630** • 6mo ago

There will be no tool as good as claude for a cheaper price tbh, the rate limits are completely justifiable IMO. I am on Max 5x and don't have issues with limits at all and use it 6-8 hours a day with 2-3 instances running.

**Acceptable-Kick-7102** • 6mo ago

The point is my usage is incosistent. SOMETIMES i use it 6-8h a day. And sometimes i don't touch it at all because i need to do other things. Its like 50:50 a month. If i was using it 6-8h a day every day (or at least most of them) in a month then yes i would pay higher price. (Yet i still see people here and on YT comments that even with MAX hitting those limits is easy).

So yeah, im still using Pro but also im looking arround. And even though im aware that Claude is one of the top ones, im ready for some compromise.

**0xFF0000** • 6mo ago

One more tip (and wondering if others find use of this too): it helps to remind CC to use subagents ("in parallel where applicable") to manage master context window, using main session more as an orchestrator. For me this is a nice in-between - should integrate well with multi-model workflows (e.g. using Codex via MCP as part of review step) if needed, but not at "external loop" level (Ralph loop style).

CC will do that for you often by default, but it's particularly useful for running investigations/research in larger source trees (not just e.g. investigating logs where it's essentially a necessity). CC is able to also search/read/grep through output of its agents (those outputs are also persisted into files automatically) if it's a lot, but either way just *approaching* the CC session as an orchestrator *regardless* of task scope is useful.

[Skip to main content](#)[+](#) Create[🔔](#) ²

Has been working for me at least.



TheLawIsSacred • 6mo ago

Reading about Claude Codes insane abilities makes me wonder: if you can spend just a *few hours* learning Claude Code CLI basically, is there any reason - whatsoever - to go back to Claude Desktop app (Chat/CC now embedded/Beta Cowork),

I ask as someone - like me - you spent hours building a. I am a non-developer yet built my own "AI Panel" with unified memory across all my web subscription frontier models.

TLDR: Most Claude subscribers think they're clever by combining it with other 2-3 browser tabs with AI models of their choice - providing the same prompt to each, and comparing outputs, all manually. Definitely, that catches some errors a single model would miss.

But for serious white collar professional work - whether legal analysis, financial review, or technical specs - capturing "some errors" isn't enough.

The above minimal approach lacks:

- persistent memory across all of your AI Panel members (you're re-explaining context every session, even if your AI has somewhat decent native memory)
- structured adversarial prompts (AI Panel models politely agree rather than constructively attack each other's reasoning and conclusions), and artifact verification (no collective AI Brain vigorously fact-checking facts, analysis, conclusions' citations against primary sources).

SO, to date, what I have done to address these gaps systematically.

What it gets me:

1. **Fast drafting:** Claude Desktop app, based on natively saved (yet adaptable as needed on the fly) scripts, orchestrates 3-6 recursive rounds with my other AI Panel members, while I make my morning coffee. Unless Claude flags a significant issue for me to review during this process, it usually just takes about 5 to 10 minutes, max, depending on the complexity of the question, and if all of my AI Panel members were "called to duty," by me that day (fyi, the total amount of AI panel members is always contingent on the complexity of the work at hand).
2. **Hard verification:** Approximately at least 5 AI Panel member-AI's adversarially check each other's work, across each round, including asking each other questions during each round, raising new ideas that are genuinely useful to my vision, etc
3. **Persistent context:** My AI Panel, thanks to a good app that auto tracks all content that I specify across my apps, now spans months of knowledge; so, without me having to re-explaining context every session - no more "as I mentioned earlier" (plus, almost of my AI Panel members are, in 2026, are getting better and better native memory).

Plus, I have a highly extensive end-of-chat "handoff" process that Claude *must* complete accurately at the end of every Claude desktop chat, including supposedly final completion of the Claude mand

Skip to main content



Create



I'm my AI Panel's manager. Claude Desktop (Opus 4.5, via Claude Max 5x subscription) handles automation/orchestration - I designated Claude Desktop app as my AI Panel's "First Among Equals."

Claude Desktop app controls my web browsers (still trialling some tools, including Playwright / Claude in Chrome / others), interacting with other AI Panel members (i.e., (ChatGPT Plus, Gemini Pro, Grok, Perplexity Pro, NotebookLM, occasionally Copilot, one or two others I forget). .

During Claude Desktop app's recursive automated orchestration, mentioned earlier, it distributes role-specific prompts to each Panel member via browser automation, collects their responses, and identifies agreement/disagreement. It flags when one member catches an error another missed and generates follow-up prompts to resolve conflicts. (Touched on some of this above, but wanted to share a bit more).

Each round, Claude produces a running synthesis that sharpens as models pressure-test each other.

By the final round, Claude delivers me consolidated output reflecting surviving consensus - or a clear map of where the Panel diverged and why.

If my Panel can't agree after 6 rounds ("Round 6 Hard Stop"), Claude Desktop also automatically escalates to me.

What makes it work, IMO:

Memory & Context

- **Unified memory layer:** All AI Panel members query shared persistent memory (via a specialized PC app + local memory servers + web browser extensions including OpenMemory + each AI's native memory + Claude's regularly distributed handoff packets to other AI Panel members), so context doesn't get lost when switching models or starting new chats.

To clarify: "unified memory" means a shared external repository with retrieval capabilities - local files, a memory database, structured notes - that all models can query on demand. Not shared weights, not magical cross-vendor brain fusion.

Infrastructure & Automation

- **MCP infrastructure:** About 12 MCP servers including filesystem, memory, Google Drive, PDF tools, Playwright (browser automation), Context7 (live documentation), desktop-commander, GitHub, Windows-MCP and others I forget right now but are useful.

Almost all my MCPs are managed through lazy-router - a hierarchical proxy that reduced startup context overhead by lazy-loading tools on demand instead of dumping dozens of tool definitions at startup.

(Note: Anthropic released their own "Tool Search Tool" in November 2025 achieving similar context savings. I built lazy-router before this existed.)

- **MCP Proxy:** I use a browser extension gateway exposing my local MCP tools to browser-based AI Panel members like a Gemini Pro and Grc to MCPs. They get read-only visibility into n sees, without write access.

[Skip to main content](#)

Create



- **Browser automation:** Claude Desktop can read/write to my PC's files, query long-term memory via several means, and interact with Chrome or Edge browser tabs where other AI Panel web-based members live.

As mentioned earlier, I use Claude for Chrome - a recent browser agent that can read pages, navigate sites, fill forms, and execute multi-step workflows across tabs, integrating directly with Claude Code.

Gemini in Chrome is also top-notch, although lacks some of the egentic capabilities of the aforementioned, at least for now. But it is very smart.



TheLawIsSacred • 6mo ago

Claude Code & Subagents

Claude Desktop now includes Claude Code (Anthropic's terminal-based agentic coding tool) built into the interface.

Claude Code CLI, however, offers things my cross-vendor Panel doesn't:

- **Native subagents:** Specialized AI subagents with persistent instructions - like a "meeting notes processor" or "document reviewer" I spin up without rebuilding prompts each time.
- **True parallel execution:** Multiple subagents run simultaneously on different aspects of a problem, tighter and faster than browser automation latency.
- **Plan Mode:** Claude Code asks clarifying questions upfront and drafts a plan.md file before executing, making complex workflows more predictable. (Edit: I read this may have come to Claude desktops version of CC recently?).

External Integrations

- **NotebookLM integration:** My Chrome/Edge web browser extensions let me push content directly into Google's NotebookLM notebooks for audio summaries, Q&A, or cross-document analysis.
- **Transcription layer:** Otter AI handles meeting/voice transcription, feeding structured notes into the memory layer. (Utility tool, not a Panel member.)
- **Writing polish:** Grammarly catches mechanical errors before final review. (Also a utility, not a Panel member.)

Governance & Quality Control

- **Structured response format:** Every AI Panel member *must* deliver: position, confidence level with reasoning, dissent from majority (with evidence), blindspot check, and at least 3 novel suggestions not yet raised.

Every response requires a "Potential Embarrassment" self-audit: "Would you be embarrassed if the user or another Panel member found this sloppy?" If yes, revise before submitting.

Every response must conclude with "Justification of Continued Panel Membership" - articulating why that contribution earns continued prese

Skip to main content



Each must catch others' mistakes. Falsification-first: try to kill your own analysis before submission.

Important: This isn't majority-vote consensus. The adversarial prompts instruct each model to attack others' reasoning. For citations/factual claims, I require primary-source verification. The goal is error detection through structured disagreement, not "three models said it so it must be true."

- **Archaeology Gate:** Before proposing new tools, Claude checks what's already been built in prior sessions - searches handoff logs, to various memory tools, filesystem. Prevents duplicate effort and institutional amnesia.
- **Approved ≠ Implemented:** Nothing "works" until empirically verified. Speculation gets killed fast and documented.
- **Grading + governance:** Every Panel member grades every other member (A+ to F) - including Claude grading itself.

Claude has authority to recommend "personnel action" based on performance. Sample recent Claude commentary to ChatGPT Plus: *"You wrote 800 words about why everyone should write 500 words. The irony is not lost."*

Gemini Pro was recently on a formal Performance Improvement Plan - barely survived after I threatened to cancel my subscription. (This aligns with what many Redditors have noticed: Gemini 3 Pro consistently fails to follow instructions in prompts or custom instructions/Gems.)

- **Reproducibility:** Every session closes with a tailored handoff protocol across multiple save targets (filesystem, native memory, markdown logs). Full traceability - no more "what did we decide last week."

Cost: ~\$300/mo total (Claude Max 5x, ChatGPT Plus, Gemini Pro, Perplexity Pro, Grok, Otter AI, Grammarly).

But I now work about 50% less than I had to at my same white collar job.

No API pricing, no token counting - I'd get slaughtered on API given my daily volume.

Time to build: 150+ hours over ~6 weeks (PowerShell, JSON schema orchestration, MCP configs, integration plumbing), though much was watching Claude Code work.

Why so long? Beyond core architecture: debugging ARM64 compatibility (Surface Laptop 7's Snapdragon X Elite breaks many x64-assumed tools), rewriting governance protocols until they are stuck, building the handoff system, and iterating through dozens of failed MCP configurations.

Plus constant refinement given how fast AI moves.

TLDR: Single-model reliance is how you get hallucinations that slip through. Multi-model adversarial checking catches what any one system misses.

Anyone else running multi-model setups v built.

[Skip to main content](#) **Create** ²

This was really helpful. Thanks



TheLawIsSacred • 5mo ago

Thanks. I appreciate that you actually took the time to read and understand how powerful a setup like this is.



specific_account_ • 6mo ago

Man, what are you doing? This is just bloat. Nobody is gonna read this.



Fair_Advantage4942 • 6mo ago

I've experimented with Superwhisper and the others you listed, but I'm very happy with aidictation.com because it automatically handles the filler words and formatting so I can prompt Claude at full speed without the editing tax.



septemous • 6mo ago • Edited 6mo ago

This is great! I added a "Topic" Hook for my status line to always 1. Remind me what i'm doing in a Terminal window. (Embarrassingly, sometimes I have multiple ones open, and I forget what I'm trying to accomplish in any specific one!)

I chose to do it as a hook so It pushes me to be intentional about that session.

Copy and past the entire text of the Gist into your terminal window:

<https://gist.github.com/Ethros19/c10c502b9bc435aeae4cb64f9a0d8aa8>



ConsiderationHour710 • 4mo ago

 Top 1% Poster

Do you have a link to your voice transcription app? I've tried myself with a voice transcription app for recording meetings using OpenAI's whisper model but found it to not capture words that well



rentprompts • 2mo ago



mnewiraq • 1mo ago

Golden post



Create



thomascherickal1 • 6d ago

This was incredible. Simply incredible. Please post on HackerNoon and LinkedIn! There's a wide world of Claude Code users there who need to read this!



mylifeasacoder • 6mo ago



Help lead our community

Apply to be a moderator

Dismiss

Apply